



# Hyonah Lim

임현아

Full-stack Developer · Seoul, Korea

I build the AI analysis server and web platform for a SIEM security product.  
3+ years building and operating web services · B.S. in Cyber Security

mmyonaa.github.io   apddfhsajrwk@gmail.com   github.com/mmyonaa  
linkedin.com/in/hyona-lim

## ABOUT

A developer with a cyber security degree and 3+ years of building and operating web services. I work primarily in TypeScript/JavaScript and Node.js, building REST APIs and the integration between servers and web clients. I currently develop a Fastify AI analysis server and a Next.js web platform for a SIEM security product at the PoC/demo stage, owning the multi-agent analysis pipeline, the LLM gateway, and the REST/SSE integration between them. Before that I built and operated a global eSIM payment and provisioning service and its airport kiosks.

## SKILLS

|                       |                                                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Backend · Data        | TypeScript · JavaScript · Node.js · Koa · Fastify · TypeORM · Prisma · PostgreSQL · OpenSearch · Kafka · OCSF · Redis · BullMQ |
| AI                    | LiteLLM · AI SDK · Multi-provider LLM                                                                                          |
| Infra · Collaboration | AWS EC2 · S3 · SES · Nginx · Docker · PM2 · Grafana · Jest · vite-ssg · Git · GitHub Actions (CI/CD) · Jira · Confluence       |
| Frontend              | Vue 3 · React · Next.js · Pinia · vue-i18n · Tailwind CSS · SCSS · PWA                                                         |

## EXPERIENCE

Inbridge   인브릿지

Sep 2025 – Present

Full-stack Developer   |   Team Under 10   |   Stage PoC · Demo

I own both the AI analysis server and the web platform of a **SIEM security product**. I defined the analysis output schema and the SSE streaming contract myself and implemented both sides, connecting AI analysis results through to the SOC console and its reports. Development tickets, sprints and long-range schedules are managed in Jira and Confluence, with progress and discussion recorded on each ticket. Ticket IDs are tied to Git branch names so code changes stay traceable to the work that caused them.

SIEM Security AI Analysis Server

Sep 2025 –

A Fastify AI server that analyzes SIEM security incidents in real time — automating the SOC analysis workflow from triage and prioritization through IOC · MITRE ATT&CK analysis, threat scoring, and compliance (ISMS-P · ISO 27001 · GDPR) and report generation. I solely designed and built the **multi-agent analysis pipeline** and the **LiteLLM gateway with per-tenant model management** that funnels every LLM call through a single layer. Using **real data collected from security vendor XDRs such as SentinelOne · CrowdStrike · Cortex**, I put together an end-to-end demo running detection → AI analysis → response → report, and delivered three product presentations to prospective customers in Korean and English.

- **Collected data → automated analysis pipeline** — integrated **OCSF** data from CrowdStrike · SentinelOne · Cortex XDR with the Kafka/OpenSearch ingestion layer, and built the multi-agent analysis pipeline that runs downstream of the Redis queue.
- **Multi-agent orchestrator (8 role-specific agents)** — Triage classifies and prioritizes each incident pulled off the queue and decides the execution scope up front (low-priority incidents are pruned); code-based agents (IOC · MITRE · Network) then run in parallel to gather evidence first, the LLM agent (SecurityIntel) is invoked only when the verdict is still unclear, and Correlation consolidates at the end → separating evidence collection from inference keeps the reasoning auditable while structurally containing per-incident cost by spending LLM calls only where needed
- **Structured output (zod schemas)** — analysis results are schema-locked into a kill-chain graph, a threat score (0-100), and a confidence value → the kill-chain graph and reports in the **SIEM SOC web platform** below consume them directly with no post-processing, and output schema validation reduces the chance of format mismatches
- **SSE real-time streaming** — progress and partial results are streamed so analysts can follow an analysis instead of waiting for it to finish (a manual re-analysis API carries the prior analysis context). Compliance and multilingual translation are split out into background enrichment agents so they never delay the core analysis response
- **LiteLLM gateway consolidation** — introduced a single proxy layer that every LLM call passes through, collapsing per-provider SDK branches into an OpenAI-compatible path and controlling fallback models, concurrency and token budgets in one place → automatic fallback is configured for provider outages, and adding supported models or providers became a configuration change. Per-tenant model management and **BYOK** (a customer's own LLM key) are isolated by virtual key allowlists that block cross-tenant access

- **PoC/demo environment observability** — set up Grafana dashboards to watch errors and traffic across the AI server and web platform, and to check the health of deployed PoC/demo environments against metrics

Fastify TypeScript AI SDK LiteLLM Multi-provider LLM Kafka OpenSearch OCSF Redis BullMQ Prisma PostgreSQL Grafana

## SIEM Security Operations Web Platform

Sep 2025 –

The **user-facing surface** that turns the AI server's analysis results into an actual SOC console — a Next.js 15 full-stack web app for a multi-tenant SIEM security platform, covering security operations end to end: per-tenant event monitoring, incident response, and AI analysis report authoring and export. It is an App Router setup that keeps UI and server components, API routes, and the Prisma multi-schema (PostgreSQL) data layer in one app. I participated as a major contributor on a team-built codebase, developing several domains — incidents, reports, endpoints — from the screen down to the API. Of these, **I built the BlockNote-based AI report editor on my own.**

- **AI report editor, built solo (BlockNote)** — implemented a flow where AI edit suggestions are reviewed as block/table/character-level diff highlights before being applied, so analysts verify rather than trust an AI draft while finishing a customer-facing report. Multilingual (KO/EN/JA) reports and PDF · DOCX · CSV/XLSX export are all completed inside the editor
- **SIEM dashboard · incident and alert management** — severity and MITRE ATT&CK analysis views, endpoint scanning and network isolation, and threat intelligence, so an analyst's detect → decide → respond path stays in the web app
- **Kill-chain visualization** — renders the AI server's structured output directly as an XY Flow · dagre topology graph (MITRE ATT&CK) so a textual analysis reads as an attack flow at a glance, with Recharts dashboard charts summarizing the metrics
- **AI server integration** — heavy AI analysis and report generation are delegated to the AI server (REST · SSE) while the web app stays light, receiving only progress through an SSE live dashboard; event search and aggregation run on OpenSearch
- **Background jobs · scheduling** — BullMQ · Redis workers and crons automate data retention, notifications and recurring reports, keeping long-running work off the request path

Next.js React TypeScript Tailwind CSS shadcn/ui Prisma PostgreSQL BlockNote Recharts XY Flow BullMQ Redis OpenSearch

## Fampy Inc. 팜피 주식회사

Jul 2023 – Sep 2025

Full-stack Developer | Team With design & dev teams | Stage In production

I handled frontend and backend development for the eSIM commerce and kiosk products and the APOC SaaS platform. I worked with the design team to build and refine service screens, and ran several projects jointly with other developers. Tickets and schedules were managed per project in monday, with Git as the shared version control across the dev team.

## eSIM Platform — Service · Kiosk · Admin

Nov 2023 – Sep 2025

I was a core developer across the whole eSIM platform — the online service, the kiosks, the admin console, and the shared middleware behind them. I took the service from development through on-site operation, handling roughly 100 real paid orders a month across web and kiosk combined.

- **Global eSIM commerce** — led the frontend design for multi-PG payments (Paygent card in USD plus Google Pay · Apple Pay in JPY) and a checkout flow shared by web and kiosk, through to the flow that provisions an eSIM from the supplier immediately after payment and delivers the activation QR and receipt as a multilingual email (the email is the delivery channel)
- **Shared backend middleware** — led the Koa · TypeORM · PostgreSQL development: hierarchical role permissions (RBAC), settlement for advertising and referral partners, and API integration with two eSIM suppliers (a domestic carrier and an overseas eSIM provider) covering catalog lookup and sourcing through order placement, dealing directly with spec differences, authentication, and character-encoding issues
- **Unattended kiosks** — built the kiosk UI selling eSIM products for 30+ countries as a Vue 3 PWA, wrapped in an Android (Kotlin) WebView app to run on the terminals. **Four units were deployed across two airports, Narita and Haneda, and ran unattended for about a year.** I led the web development and improved the WebView app's hardware integration (receipt printer, payment terminal) and multilingual encoding issues, then covered early operations and real-time on-site issue response in Japan after launch

Vue 3 TypeScript Pinia vue-i18n PWA Koa TypeORM PostgreSQL Kotlin AWS

## APOC Platform — Play · Studio

Jul 2023 – Aug 2025

- **Platform renewal** — a full interface overhaul and component-based refactor that improved maintainability and extensibility
- **Payment system** — Toss · PayPal integration (one-off and recurring subscriptions) and the PayPal checkout flow design; worked through the existing payment/subscription/cancellation history table structure to implement the payment features (a unified record across multiple PGs), plus auto-renewal batches and Jest tests
- **Tablet support for the authoring tool** — improved touch UX (establishing the 768-1023px baseline) and designed 61 test cases across 6 devices × 6 browsers, surfacing and tracking 39 issues

Vue.js Koa PostgreSQL Jest SCSS

FREELANCE

BK Theater — Promotion & Venue Rental Site | Team Solo | Stage Live (launched Nov 2025) Apr 2025 –

A site covering performances, notices and press releases, venue rental information, and admin content management (bktheater.com, launched Nov 2025), which I built solo from planning and design through development, deployment and ongoing operation.

- **Service & API** — built the public site and admin features with Vue 3 · TypeScript and Koa · PostgreSQL, implementing CRUD for performances, notices, press releases and banners, a Quill editor, and S3 file uploads.
- **Architecture & deployment** — separated backend responsibilities into routes-controller-service-repository, and set up vite-ssg-based SSG with dynamic meta tags and a sitemap for search visibility. Deployed on AWS EC2 with Nginx and PM2.
- **Operational improvements** — fixed data loss on partial edits and a view-count refresh issue, and consolidated 20 public and admin views into 10 shared pages, cutting roughly 4,000 lines of code.

Vue 3 | TypeScript | vite-ssg | Pinia | Koa | PostgreSQL | AWS EC2 | S3 | PM2 | Nginx

PERSONAL PROJECTS

blog-mcp · Blog Publishing MCP Server | Team Solo | Stage Live (unattended publishing) Jul 2026 –

A personal project where I learned the Model Context Protocol by building an unattended publishing pipeline. The stdio MCP server and an Astro blog live in a pnpm workspace monorepo, and it has published 100+ posts across 6 sections without human intervention.

- **Hand-built stdio MCP server** — exposes all three primitives (Tool · Resource · Prompt) from a single server
- **Separation of model calls from tool execution** — responsibilities split across the trigger, the orchestrator, and the MCP server
- **Deterministic publishing gate** — validates that follows/related entries and in-body internal links actually exist and are well-formed, blocking link hallucination
- **Unattended publishing pipeline** — a daily cron runs the orchestrator so an LLM agent picks a topic, writes, and self-reviews, with a PreToolUse hook validating the publishing rules before anything ships
- **View-count-driven topic selection** — page views feed back as a weight on the next topic choice (with a cold-start gate and graceful degradation)

TypeScript | MCP | @modelcontextprotocol/sdk | zod | Astro | Supabase | GitHub Actions

daily.quiz · Certification Practice Site | Team Solo | Stage Live Aug 2026 –

A static practice site for two Korean national IT certifications, with multiple-choice written questions and short-answer practical questions graded instantly. A sibling project to the blog-mcp blog: each explanation links through to the corresponding concept post.

- **Auto-grading short answers** — absorbs the hard-to-judge cases that come from having no answer choices, by normalizing notation variance (case, spacing, full-width characters, hyphens) and keeping a per-question list of accepted spellings. Only descriptive answers, which cannot be automated, are explicitly drawn out as self-graded
- **Two exams on one set of screens** — questions inherit their exam through their topic, so routes are shared via rest parameters and only the grader, the subject areas, and the storage key branch per exam
- **Content quality enforced by the build** — 875 written questions, 67 practical questions and 137 concept topics are LLM-generated from the blog's concept posts, while a zod publishing gate fails the build on schema violations or references to non-existent topics
- **Continuous sync with the blog (weekly cron)** — coupled to the sibling blog through RSS and the GitHub tree alone, with no API; a missing linked post fails the build. A weekly cron compares blob SHAs and files issues for dead links, revised posts, and topics not yet covered by a question

Astro | TypeScript | zod | Supabase | RLS | GitHub Actions

ACTIVITIES

Product Presentations to Prospective Customers May – Jul 2026

- **Three SIEM product demos and presentations** — a global IT company (Korean, May 2026) · an enterprise affiliate (English, Jun 2026) · a major Korean enterprise (Korean, Jul 2026)

RSAC 2026 Mar 2026

Attended the RSA Conference — research on current security trends and solutions

TEdI Coding Academy, Gwanggyo · Teaching Assistant and Instructor Aug 2022 – Jun 2023

Taught C/C++ algorithms and data structures, and introductory Python programming

EDUCATION

Ajou University · B.S. in Cyber Security Mar 2020 – Feb 2025

Cloud/Bigdata security track · GPA 3.96 / 4.5

LANGUAGES

Korean (native) · English — professional working proficiency, including product demos and presentations in English